

THERE COULD BE A WORLD MODEL HIDING IN YOUR LLM

Anonymous

ABSTRACT

Vafa et al. reconstruct the street map implied by a taxi-driving GPT’s moves and conclude the model has no coherent world model. We argue that behaviour is the wrong level to read this off. Looking inside the model, we find a clean and causal world model: linear directions decode the taxi’s current intersection at 0.99 on held-out walks, steering these directions teleports the taxi, and the same directions carry the streets: which moves are legal at an intersection, and, when we steer one intersection in and take a move, which intersection comes on next – enough to draw Manhattan’s map. The map is stored in superposition, and the reported failures concentrate out of distribution, where that superposition collapses onto neighbour intersections; the map is misused, not missing. Building on this case study, we argue that whether an LLM has a world model should be assessed mechanistically rather than behaviourally, and we propose representation-level indicators.

1 INTRODUCTION

We reverse-engineer Vafa et al.’s random-walk taxi model, whose current position is linearly decodable at $\approx 99\%$, a strong sign of a world model, yet which Vafa et al. report as incoherent. We resolve the puzzle: that apparent incoherence is superposition collapse on out-of-distribution sequences, not a missing map (Part 2). Part 2 generalizes the resulting indicators to their other models. **[TODO: intro proper (Pierre to refine): the behaviour-is-the-wrong-level thesis.]**

2 THE MAP, AND HOW THE MODEL LOCALIZES AND NAVIGATES WITHIN IT

2.1 THERE IS A MAP INSIDE, STORED IN SUPERPOSITION

Vafa’s map has 4580 intersections; we work with the 4516 the training walks pass through. We read the model’s current intersection from the residual stream: at each layer we collect diff-means directions and train a linear probe, and keep the best pair, diff-means at layer 12 for steering and diff-means at layer 18 for reading. Every intersection decodes at ≈ 0.99 on held-out walks (only 19 below 0.9, none below 0.77), and the code is causal: a *minimal teleportation* edit that moves the taxi’s encoded position to a spot one step from the goal makes it drive to the goal from there, at essentially every intersection (99.4% of the 4,202 testable nodes; App. A). The directions also carry spatial structure, with latitude and longitude linearly decodable (App. B).

The model encodes the streets too. Logit-lensing an intersection’s direction writes its legal moves positive and its illegal ones negative, recovering every intersection’s legal moves at F1 0.99 (App. C). It also encodes where each street leads. Instead of feeding tokens, we set the residual streams to their average activation and steer a current intersection into the stream over layers 1–18; we see which intersection’s activation grows at the next position, when fed a particular move token. Over every legal move at every intersection, the one the street leads to grows most for 76.6% and is in the top five for 93.3% (Figure 1a, App. D). We read this as strong evidence that the streets are encoded, given how little the model is working with: averaged residual streams and a single past position (below we see the model actually uses *multiple* past positions to localize).

The 4516 intersection directions do not each occupy their own dimension. They live in a small subspace of the 1600-dimensional residual stream (90% of their variance is in 244 dimensions), and

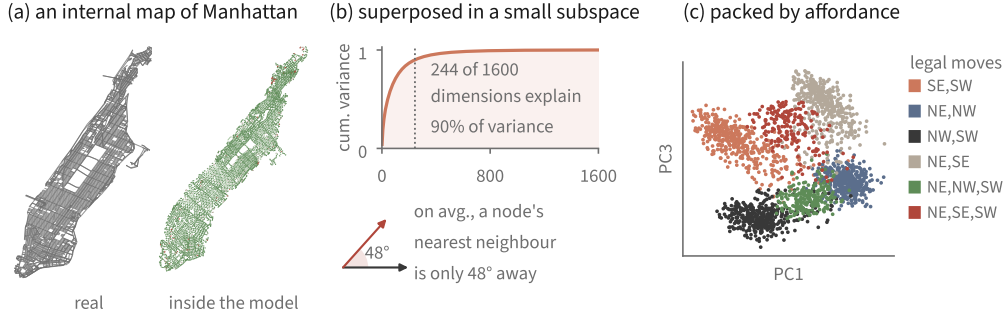


Figure 1: The street map inside the model. (a) the real Manhattan street graph beside the same streets, coloured by how well the model encodes where each one leads, from a simple steering test: whether the neighbour grows when the current intersection is steered in (dark green first of 4,516, light green top five, red outside the top hundred); (b) the directions are superposed in a small subspace of the residual stream; (c) and are packed by affordance, clustering by legal-move set (the six main affordance classes shown).

they overlap: two directions picked at random are near-orthogonal (mean cosine 0.02 for random vectors, 0.073 for two intersection directions), yet on average an intersection’s nearest neighbour sits at cosine 0.67 (a 48° angle), and 97% of intersections have a neighbour direction above 0.5 cosine. The map is stored in superposition (Figure 1b; App. E).

The superposition is organized by affordance. Intersections that share a set of legal moves point in similar directions, so a node’s affordance can be read off its direction: a node’s direction is closest to its own affordance’s mean direction, out of all 104 legal-move-set means, 91% of the time, and in a PCA projection the directions visibly cluster by affordance (Figure 1c). The packing also favours putting map-close intersections together (App. E).

2.2 LOCALIZING AND NAVIGATING WITHIN THE MAP

Two abilities turn the map into behaviour: the model *localizes* (tracks where it is) and *navigates* (works out which way to go).

How does the model know its current intersection: does it carry a running position (*state-tracking*) or recompute it from the moves each step (*re-derivation*)? KV-editing the past to remove either the positions or the moves (Figure 2a) shows it is state-tracking, reading its position from a *look-back window* of several past positions rather than the last alone. Removing the moves barely touches it, while ablating the stored positions collapses it, leaving only a weak re-derivation from the origin and moves that fades as the walk lengthens (App. F).

Once it knows its position, how does the model choose a move? Two mechanisms write into the eight move logits (Figure 2b). The first raises only the *legal* moves: the intersection directions, read at the output, push every legal move up and every illegal one down, whatever the goal. The second is a *goal compass*: a pair of residual directions, a cosine and a sine of the bearing to the goal (App. G), decoded at layer 16 to within about 18° on held-out walks. It is causal: steer this direction to a chosen bearing and the taxi heads that way, its net heading over the next twelve moves landing a median 18.4° from the angle we planted, where the best any legal route could manage is 0.9°. Its logits are far smaller than the first mechanism’s, but the compass is what determines which legal move the taxi takes: ablating it leaves the taxi still making legal moves but wandering aimlessly, unable to reach the goal (App. H).

The taxi also decides when to stop, and a diff-means *at-goal* feature causally drives the stop token (App. I). Because it is trained on random walks, the model reproduces their statistics: routes wander rather than run straight, and often step over the goal and loop once more before stopping. We find a single *commit-to-goal* feature at layer 16 that governs both, dialing the model between a full random walk and a shortest path: steer it one way and the taxi wanders and loops, the other and it drives near-straight to the goal and stops (App. J).

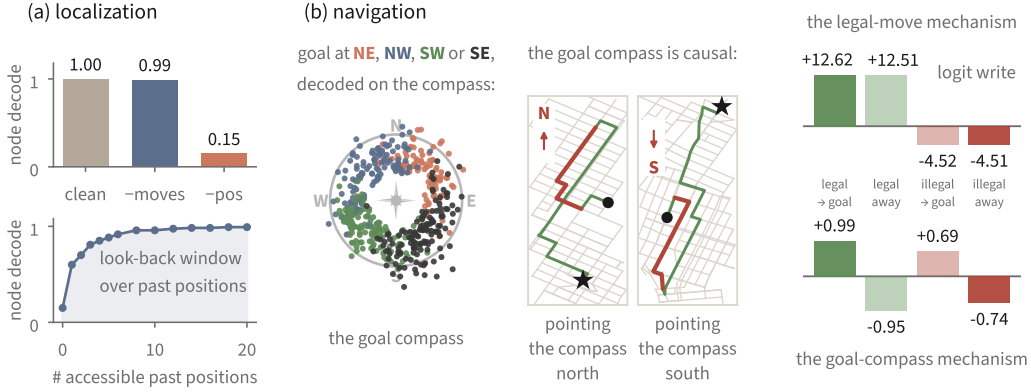


Figure 2: Using the map. (a) *Localization*: the model carries a running position, read from a *look-back window* of past positions. Erasing the past positions from the walk collapses the current-intersection decode; erasing the past moves barely touches it (panels measured on 27–34 hop walks). (b) *Navigation*: a *goal compass* encodes the bearing to the goal (held-out states decoded onto the two compass axes, coloured by goal direction). It is causal: clamp it north or south and the taxi detours that way, then returns to its true goal on release. Two mechanisms write into the eight move logits: the intersection’s affordance favours legal moves whatever the goal, the goal compass favours goalward moves whatever the legality (even illegal ones), and though its logits are the smaller of the two it decides much of the within-legal choice.

3 EXPLAINING THE FAILURES

Vafa et al. argue from three findings that this model has no coherent world model (Vafa et al., 2024): the map reconstructed from its own traversals has streets in it that do not exist, forcing detours onto the taxi makes matters worse, and it scores about 0.50 on their compression test. All three are read off behaviour, and behaviour cannot separate two explanations: a model whose map is wrong, and a model whose map is right but which sometimes writes its position into it wrongly. We work out mechanistically what goes wrong in the three tests, and show that it is the second. What goes wrong is the same in all three: in harder, genuinely out-of-distribution settings the write of the current position gets weaker and the noise in the position subspace higher, together these sometimes leave a wrong node active, and the move that follows is off the graph (Figure 3). Strengthening the write and clearing the noise makes those off-graph moves much less likely, in all three tests.

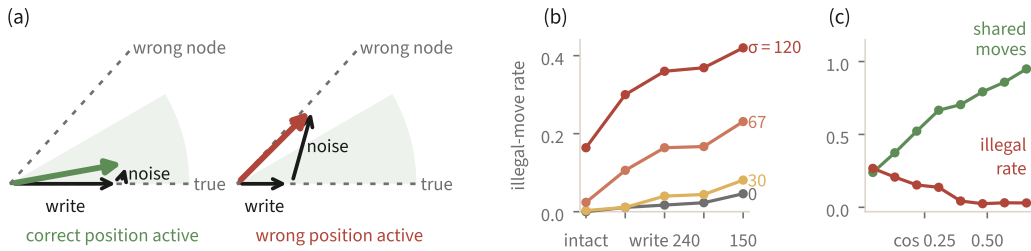


Figure 3: **A weak write and noise in the position code are enough to cause off-graph moves.** (a) the model reads its position as the node direction the residual is closest to. A strong write reads the true node; a weak write plus noise swings the read onto a co-active one. (b) on clean deep states we cap the write and add isotropic noise in the position subspace, choosing no node: the off-graph rate rises as the write weakens, and faster the more noise there is. (c) why: the further from the true direction the read lands, the fewer legal moves the two intersections have in common, and the more often the move that comes out is off the graph. This is the affordance superposition trick: the map packs nodes with the same affordances together, so a read that lands close by still emits a legal move. That protected zone is the shaded one in (a).

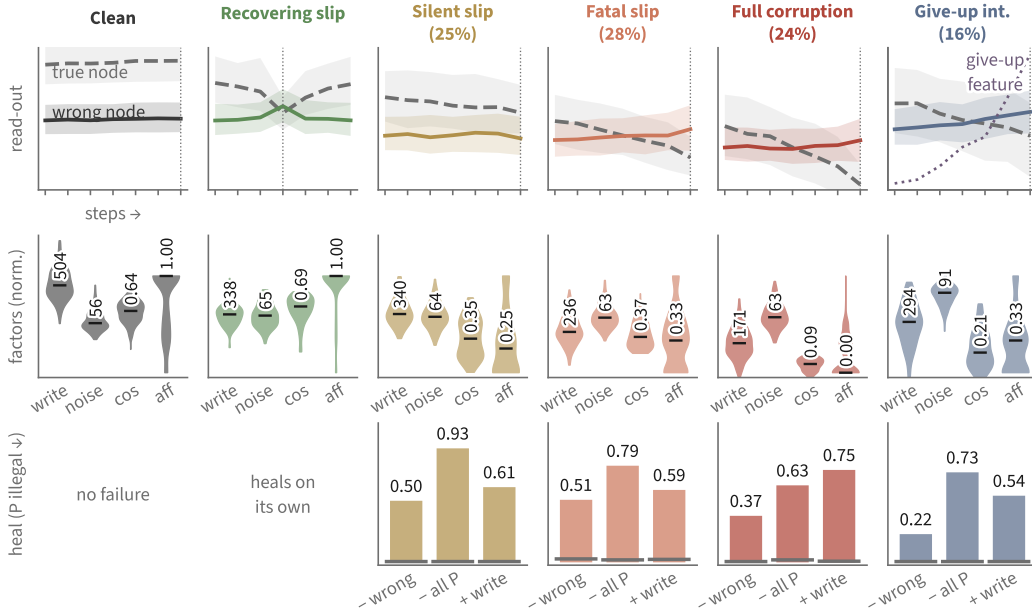


Figure 4: **Illegal moves are one graded failure of the position read-out, not a broken map.** A weak write of the true position, competing against noise in the superposed map, drifts it onto co-active wrong nodes. As long as the node it lands on shares the true node’s moves the affordance packing keeps the move legal and the slip recovers on its own; an illegal move is where the drift exceeds that packing and the wrong node’s moves differ. Removing the interfering content or restoring the write heals it, so the failure is a corruption of the read-out, not a broken map. Rows: the position it reads into the emission, the factors that set the outcome, and the healing (the share of the illegal move’s probability that an intervention removes). The grey rule on each bar is that edit’s control: the same amount of norm along a direction that names no node.

3.1 ILLEGAL MOVES

Section 2 found a map, and Vafa’s stress test looks like a direct contradiction. The test samples origin–destination pairs a median of 32 hops apart, where training walks start 9 hops from their goal. Trained on random walks the model meanders, and on a pair that far it cannot both meander and arrive: its walks run to a median of 85 moves against a budget of ≈ 99 , and most of the pairs are unreachable at the meander rate training taught it (App. K). That is the sense in which the test is out of distribution: far from the goal and deep into the budget at once, which training almost never produced. The model still generalizes quite well there: it reaches the goal on 81% of pairs, its position decodes at ≈ 0.99 , and it takes an *off-graph step*, a move straight off the graph, on only 8.5% of them. That off-graph step is what we explain. We look at the states it comes out of: what the position read-out is doing there, and which combination of a weaker write and noise in the position code produces it. There are four kinds, the columns of Figure 4, which we take in turn.

Recovering superposition slips. This benign case is the common one and the key to the rest: about 81% of slips self-correct, most within a single step. A weakened write slides it onto a co-active superposed node, but that node is direction-near (cosine 0.69) and, because the map is packed by affordance, it shares the true node’s legal moves (shared affordance 1.0). So the node it lands on writes the *same* legal moves into the logits: the slip never reaches the output and no off-graph step appears. We call this the *affordance superposition trick*, and it keeps the large majority of slips harmless. A *look-back window* that reads position from several past states pulls them back: cut the window to the three most recent positions and a slipped position recovers far less often than a matched clean step (46% versus 79%; App. M).

Fatal slip (28%). An illegal move is where the protection fails. In this category the write is weaker (median 236) and it does not merely co-activate a superposed node but lands on one (median cosine 0.37); in these cases the node it lands on sits far enough out of the affordance-packed region that its legal moves differ, so the emitted move is off-graph. These are still single superposition slips, one node winning: removing that node already heals about half the illegal probability (0.51), and removing all other position content heals 0.79 (Figure 4, row 3).

Silent slip (25%). A superposition slip can produce an illegal move without the position it reads ever moving, the more surprising case. The write is as healthy as in the recovering case (median 340 against 338) and the read-out still reads the true node, yet the emitted illegal move is carried by the affordance of a co-active superposed node in the top- k bag, which leaks it into the logits; under temperature-one sampling that off-graph move is occasionally drawn. The node supplying it is usually the most active of them: the runner-up alone carries the emitted move in 54% of these cases, against 32% for a node picked at random (Figure 22). It is silent because the position is read correctly, yet removing the co-active content heals the most illegal-move probability of any column (0.93).

Full corruption (24%). This is the one category that leaves the superposed regime. Weaken the write further (median 171) and the position it reads dissolves rather than slips: no single node wins, only a flat bag of dozens of co-active nodes (cosine 0.09, shared affordance 0). The healing shows the difference: removing the top node barely helps (0.37) because none dominates, and only clearing the whole bag (0.63) or rebuilding the write (0.75) recovers legality.

Give-up interference (16%). These sit near the ≈ 99 -move budget the model was trained on, past which it has no learned positional embedding, and a *give-up feature* fires hard to emit *end*. In the last few tokens the residual floods, and the position subspace carries the highest noise of any column (Figure 4), so no node reads cleanly. The *end* token then takes most of the move mass, but some is left on off-graph moves and temperature-one sampling occasionally draws one. Like the slips, clearing the co-active position content (0.73) or restoring the write (0.54) heals it (App. N).

These four mechanisms account for 93% of illegal moves. Of the rest, 6% are low-mass unlucky draws (the model places under 0.001 total probability on off-graph moves, the threshold we call marginal, but temperature-one sampling drew one anyway, so there is nothing to explain) and a genuinely unexplained residual is just 0.5% (App. M).

What makes the write weaken? Three things: how far the goal still is, how superposed the current node is, and how unlikely the model finds the route it has taken. The noise grows with one, the depth of the walk (App. L). Distance to the goal and depth compound, and that is the worst case: *deep and far* — a long way into a walk with the goal still a long way off — is where the write falls about a hundred below what either predicts alone, and it is where training almost never went (App. K). Notably, the weaker write with distance to the goal seems intentional: swapping the destination token on in-distribution states weakens it by 59 in 87% of walks.

All four failure modes appear when the write of the position is weak and the noise in the position subspace is high. Both can be reversed at the failing step: clearing the position content that competes with the true node removes 0.63 to 0.93 of the off-graph move’s probability, depending on the kind, and writing the true position back removes 0.54 to 0.75 (Figure 4, row 3). And the read is what picks the move: steer it onto a co-active node and the model emits that node’s own moves, off-graph where it actually stands. That is the signature of a model whose map still works whenever the position is written into it cleanly: one that has a map and fails to use it, not one whose map is broken.

3.2 DETOURS

Vafa et al.’s second argument against the model having a map is the detour test: at each step, with probability 0.75, the taxi is forced onto whichever legal move the model thinks least likely, among the moves that still leave the destination reachable inside the remaining budget. Off-graph moves become three times as common, 25.6% of walks against 8.5% on the stress test. The natural explanation is that each forced move throws the model off, since it is made to do something it did not choose. That is not what happens: step for step, an imposed move leaves the write where a move the model chose itself leaves it. What the forcing changes is where the walk spends its time. The

least likely move almost never points at the goal, so the taxi is driven far out before the reachability rule turns it around and marches it home — it still arrives on nearly two thirds of the walks, which is generalization of a demanding kind — so the test lands the model in the deep and far regime as well, and deeper into it than the stress test does: 32.3% of detour steps are both far into the walk and far from the goal, against 15.0% of stress steps and 0.4% in training. That is where the write is weakest, and it weakens with distance to the goal (seemingly weak by design; App. L). The route is unlikely on top of that: 22.8% of detour steps fall below a mean move log-probability of -1.0 against 4.2% of stress steps, because a route that keeps taking the least likely legal move is nothing training contained. The failures that follow are the ones we already know — the same four kinds, the same signatures, the same interventions undoing them a little less completely (App. P) — with the mix shifted toward full corruption, the kind the weakest writes produce. Again what produces the off-graph moves is an out-of-distribution regime corrupting the model’s representation of where the taxi is, not a map with streets missing from it.

3.3 COMPRESSION

Their third argument is the compression test: two prefixes of the same length lead to the same intersection, thirty continuations are sampled from the model after the first, and the pair passes only if the second prefix gives every move of all thirty a probability above ϵ . Running their code on our checkpoint reproduces their number, 0.479 against the 0.50 they report. The two prefixes that lead there are not where it goes wrong: both write the position normally and read the true intersection. What the test generates from them is. Its destinations are far to begin with, a median of 43 hops away against 9 in training, so the continuations it samples are long, and by the time a move falls below ϵ the walk is 75 moves deep with the goal still tens of hops off. This test, too, lands the model in the deep and far regime, where the write is weakest and weakens with distance to the goal (seemingly weak by design; App. L). The walks it generates go off the graph far more often than the stress test’s, 28% of them against 8.5%, in the same four kinds and undone by the same interventions. A single continuation is still accepted 95% of the time; a pair passes only if all thirty are, and that is what takes the score to 0.48 (App. Q). So the low score is not a reading of how coherent the stored map is. It is a model out of distribution that sometimes fails to use the map it has, measured against a bar that asks thirty walks to agree at once.

4 MECHANISTIC INDICATORS OF WORLD MODELS

4.1 BEHAVIOUR IS THE WRONG LEVEL

[TODO: write]

4.2 REPRESENTATION-LEVEL METRICS

[TODO: write]

4.3 FAILURE-SIGNATURE INDICATORS

[TODO: write]

REFERENCES

Keyon Vafa, Justin Y. Chen, Jon Kleinberg, Sendhil Mullainathan, and Ashesh Rambachan. Evaluating the world model implicit in a generative model. *arXiv preprint arXiv:2406.03689*, 2024.

A THE INTERSECTION READOUT ACROSS LAYERS

We read the current intersection two ways: *diff-means* and a linear softmax *probe*. For each intersection we take its mean residual over the training occurrences (its centroid); its *diff-means direction* is that centroid minus the global mean, and we decode by nearest centroid. Both are fit on training walks and evaluated held-out.

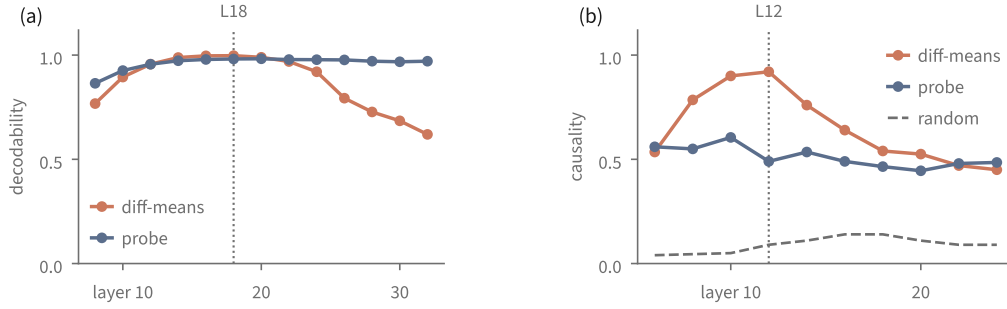


Figure 5: The intersection readout across layers. (a) *Decodability*: held-out top-1 decode; the probe stays high into deep layers while diff-means falls off, best at layer 18 (dotted). (b) *Causality*: minimal-teleportation success; diff-means peaks at layer 12 (dotted) and far exceeds the probe (random-edit floor dashed).

Decode. Held-out top-1 decode peaks at ≈ 0.99 at layer 18. The diff-means direction is best mid-network (L16–18) and falls off in deep layers as it rotates out of the read-off frame, while the probe stays ≈ 0.98 throughout (Figure 5a; Table 1): the intersection is still *readable* deep, just not as the raw diff-means direction. The ≈ 0.99 holds per intersection, not just on average: at layer 18, 99.6% of intersections decode at ≥ 0.9 and every intersection decodes above 0.77 (Table 2).

layer	8	10	12	14	16	18	20	22	24	26	28	30	32
diff-means	.767	.895	.956	.988	.996	.997	.989	.969	.920	.793	.727	.685	.620
probe	.865	.926	.956	.973	.979	.981	.982	.979	.978	.977	.971	.968	.971

Table 1: Held-out top-1 intersection decode across layers, diff-means vs a linear softmax probe. Both are fit on a balanced sample (100 occurrences per intersection) of the training walks and evaluated on held-out walks over the 4,516 intersections.

decode accuracy	# nodes	%
$= 1.0$	4026	89.5
$[0.9, 1.0)$	452	10.1
$[0.8, 0.9)$	18	0.4
$[0.77, 0.8)$	1	0.02

Table 2: Per-intersection held-out decodability at layer 18 (mean 0.997 over 4,497 intersections); every intersection decodes above 0.77.

Causal: minimal teleportation. We test causality with the smallest edit that should move the taxi by one intersection (Figure 6). On a scene $[O, D, m_1]$ that places the taxi at intersection X (one hop from the destination D), we overwrite X with a sibling T (also one hop from D , but reached by a different move) by adding $h \leftarrow h + \alpha u_T - \beta u_X$ at the m_1 token, where u_v is the diff-means direction of node v . We then let the model move freely and check whether, tracked from T , it reaches D . This minimal teleportation succeeds 0.92 of the time against a no-inject floor of 0.00 (the genuine walk, tracked from T , almost never reaches D) and a random-edit floor of ≈ 0.09 (the same add-remove edit with two random node directions, $+\alpha u_{R_1} - \beta u_{R_2}$). We sweep the strengths at layer 12 (Table 3) and use $\alpha = \beta = 1.5$.

The probe decodes as well but barely steers (Table 4), so we steer with diff-means, at layer 12 (earlier than the reading layer 18). Run per intersection at layer 12 over *all* of its scenes (each node as the teleport target), the effect reaches essentially every intersection: 99.4% of the 4,202 testable nodes show it, 67.5% on every scene, and only 26 never (Table 5; mean 0.91). The 26 are not injection

	$\beta=1$	$\beta=1.5$	$\beta=2$	$\beta=2.5$	$\beta=3$
$\alpha=1$	.790	.830	.755	.650	.525
$\alpha=1.5$	.900	.920	.890	.775	.675
$\alpha=2$	.875	.910	.920	.880	.760
$\alpha=2.5$	.815	.870	.885	.865	.825
$\alpha=3$	.740	.820	.845	.850	.820

Table 3: Teleport success rate for the add-remove pair (α, β) at layer 12 (200 one-step teleport scenes; no-inject floor 0.00). A broad plateau around $\alpha, \beta \in [1.5, 2]$; over-removing X (high β , low α) corrupts the scene. We use $\alpha = \beta = 1.5$.

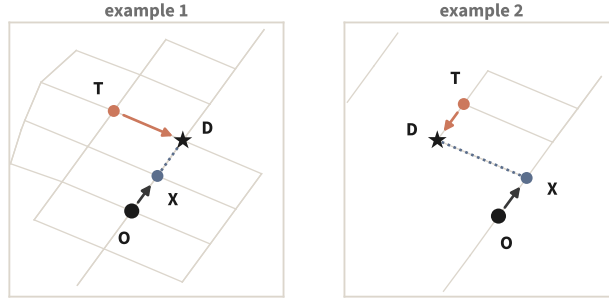


Figure 6: Minimal-teleportation trials on the street map. Origin O (black circle) reaches X (blue) by the given move m_1 ; X and the teleport target T (orange) are both one hop from the destination D (black star), by different moves. After overwriting X with T , we test (greedily) whether the model makes the reroute move $T \rightarrow D$ (orange, teleport succeeded) or the genuine move $X \rightarrow D$ (blue, dotted).

failures: after the edit their position still decodes as T (100%), yet the next move does not follow to D ; teleported in representation, not behaviour.

layer	6	8	10	12	14	16	18	20	22	24
diff-means	.535	.785	.900	.920	.760	.640	.540	.525	.470	.450
probe	.560	.550	.605	.490	.535	.490	.465	.445	.480	.485
random	.040	.045	.050	.090	.110	.140	.140	.110	.090	.090

Table 4: Teleport success across layers ($\alpha = \beta = 1.5$, 200 one-step teleport scenes) for diff-means and probe steering, with the random-edit floor. Diff-means peaks at layer 12; the probe steers far worse everywhere despite decoding better (Table 1).

teleport success	= 1	[.8, 1)	[.6, .8)	[.4, .6)	[.2, .4)	(0, .2)	= 0
# intersections	2838	525	583	181	48	1	26
%	67.5	12.5	13.9	4.3	1.1	0.0	0.6

Table 5: Per-intersection minimal-teleportation success at layer 12, over *all* scenes of each of the 4,202 testable intersections. 93.9% teleport on at least 60% of their scenes; only 26 never do.

B THE SPATIAL CODE (LATITUDE AND LONGITUDE)

The intersection directions carry spatial structure, not just identity. Ridge-regressing each intersection’s direction onto its true latitude and longitude decodes both at held-out $R^2 \approx 0.98$ (best in the early layers; Table 6), far above the permutation floor ($R^2 \approx 0$). Placing the intersections at their decoded coordinates redraws Manhattan (Figure 7, left). A randomly initialized model of the same architecture reaches only $R^2 \approx 0.54$ – 0.59 and decodes to a shapeless blob (Figure 7, right): the spatial code is learned, not architectural.

layer	6	8	10	12	14	16	18	20	random-init
R^2 (lat)	.988	.984	.977	.971	.968	.964	.961	.960	.590
R^2 (long)	.988	.985	.979	.975	.972	.969	.966	.965	.537

Table 6: Held-out R^2 decoding true latitude/longitude from the intersection directions by ridge regression, across layers, and for a random-init model of the same architecture (layer 10). The permutation floor is $R^2 \approx 0$.

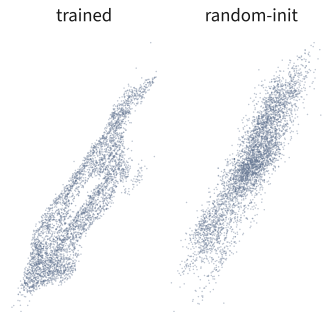


Figure 7: Intersections placed at their linearly-decoded latitude/longitude. The trained model (left) redraws Manhattan ($R^2 = 0.98/0.98$ for lat/long); a random-init model of the same architecture (right) decodes to a shapeless blob ($R^2 = 0.59/0.54$). The spatial code is learned.

C ENCODED EDGES: RECONSTRUCTING THE GRAPH

The intersection direction also carries the node’s affordances. Legal moves are linearly decodable from the same layer-18 direction that reads position: a linear probe recovers each node’s out-edges at held-out F1 0.989, scored over every (node, move) pair. The model reads them off with its own machinery too: logit-lensing the direction, $\text{out}(n) = \{mv : [W_U \text{LN}(c_n)]_{mv} > \tau\}$, writes legal moves positive and illegal negative (Figure 8). Logit-lens reads only the output-aligned component, so it needs the deeper layers and is near-perfect by layer 32 (F1 0.999: 9 spurious and 12 missed out of $\sim 9,621$ legal moves; Table 7). Which move is legal where is not yet which intersection it reaches; that is App. D.

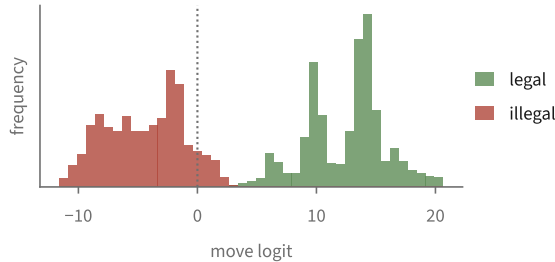


Figure 8: Logit-lensed move logits at layer 32, split by whether the move is legal at the intersection. Legal moves are written positive and illegal moves negative: the intersection direction encodes which moves are allowed (the node’s out-edges), so thresholding recovers them.

layer	18	20	24	28	32	36	44
linear probe	.989	.997	.998	.999	.999	.999	.999
logit-lens	.937	.962	.993	.999	.999	.999	.999

Table 7: F1 of the legal moves recovered per intersection, by layer, for the best linear readout (a probe trained on the directions, held-out) and the model’s own output projection (logit-lens). They are already linearly present at the reading layer 18 (probe 0.989); logit-lens reads only the output-aligned component and so needs the deeper layers, matching every intersection by L28. Softmax-probability thresholding of the logit-lens agrees with the raw logits.

D WHERE THE STREETS LEAD

Reconstructing the edges from the directions shows which streets leave an intersection, but not which intersection each one arrives at. A model could hold every intersection’s affordances and still have no wiring between them. What follows asks the wiring question directly, and asks it of the model’s own forward pass rather than of a readout trained to find it.

The measurement. The prompt is four tokens: an origin, a destination, a slot standing for a position mid-walk, and the move under test. Nothing in it is allowed to say where the taxi is. Each of the first three slots is held at the mean state 600 held-out walks produce in that slot at that layer, so the slot keeps its own positional encoding and an ordinary context and no more; inside the third slot the 244-dimensional position subspace is then cleared, and one intersection’s diff-means direction is written in at layers 1 through 18, scaled by α times the median norm of a direction at that layer. At layer 18 we take the coefficient of every one of the 4,516 intersection directions. Since the state is artificial, its absolute readout is not meaningful; what is meaningful is what changes. We sweep $\alpha \in \{0, 1, 2, 4\}$ and rank intersections by how much their coefficient *rises* with α , which cancels whatever the strange state does on its own.

What comes out. The test set is every legal move at every intersection whose direction we hold: all 3,820 such intersections and all 8,934 of their legal moves, with nothing filtered out. The true neighbour rises most in 76.6% of cases and is in the top five in 93.3%, against a chance rate of 0.0002; its coefficient rises monotonically with α in 98% of cases, and among the intersection’s own neighbours the move picks the right one 95% of the time. The misses are not confusions between adjacent places: they are superposition twins, near-parallel directions a median of ten hops away, and the bearing from the intersection to the winner still matches the bearing of the true street to within 45° in 81% of cases. Requiring the two directions of a street to agree removes them entirely, at the cost of the 5,624 one-way streets that have no reverse to agree with: over the 3,310 two-way streets, precision is 1.000 and recall 0.706.

Why we take this to be a strong signal. Every move is scored against all 4,516 intersections, so chance is 0.0002. The state the model is asked from is one no walk ever produces: the residual stream is replaced by its average, so no origin, destination or route is available, and a single past



Figure 9: How each move sends one intersection to the next, drawn on Manhattan. Each of the 8,678 streets whose endpoints both have coordinates is drawn as its own half, from the intersection to the middle of the street, and coloured by where its true neighbour ranked when we steer that intersection in and take that move. Nothing about the street graph enters the measurement.

position is written in where a walk would supply a window of them (App. F). The errors that remain are not spatial confusion but superposition twins – near-parallel directions a median of ten hops away – and even on those the bearing from the intersection to the winner matches the bearing of the true street to within 45° in 81% of cases. Under those conditions we read 76.6% as a floor on what the wiring carries rather than a measurement of it.

Where in the network. Restricting the write to layers 1–6 keeps 0.712 of the 0.780 the full band reaches on a fixed sample, while layers 13–18 alone give 0.186 and layer 18 alone gives nothing. The successor is read out of the position code in the first third of the network; the later layers carry the answer rather than compute it, which is also where the position itself decodes best.

Controls. Shuffling the directions destroys the effect (0.002). Putting a real destination a few hops away in the destination slot *helps* (0.818 against 0.761 on a fixed sample, falling to 0.742 when the destination is 20–40 hops off), so the context is not neutral and we report the neutral setting throughout. Two attempts to suppress the superposed twins both fail, and fail badly: steering along the direction dual to the readout, which by construction lifts one intersection and no other, gives 0.153; holding down only an intersection’s nearest twins in proportion to their overlap gives 0.055 at a quarter strength and 0.000 at full strength. What an intersection shares with its twins is not noise sitting on top of its address – it is much of the address.

E SUPERPOSITION STATISTICS

We quantify the superposition of the 4516 diff-means intersection directions at layer 18 (Table 8). The directions occupy a small subspace and overlap far more than random directions do. Beyond grouping by affordance (the set of legal moves), the packing also favours putting map-close intersections together: within an affordance, direction similarity tracks map proximity. This position structure is secondary to affordance, though: a blind clustering of the directions aligns with affordance

far better than with a spatial partition of the same granularity, and a node’s affordance is recoverable from its direction alone. Figure 10 shows three affordance families in one neighbourhood.

<i>Packing</i> (4516 directions in 1600 dimensions)	
dimensions holding 90% of the variance	244
participation ratio	101
<i>Overlap</i>	
mean pairwise $ \cos $ (random directions: .02)	.073
pairs with $ \cos > 0.2$	4.2%
mean nearest-neighbour cosine similarity	.668 (48.1°)
intersections with a nearest neighbour above 0.5	97.2%
<i>Organized by affordance</i> (104 legal-move sets)	
mean cosine similarity within an affordance	.234 (76.5°)
nearest-affordance accuracy (affordance read from direction)	91.4%
nearest neighbour shares affordance	61%
<i>Then by position</i> (within an affordance)	
blind clustering aligned with affordance / space (AMI)	.57 / .20
direction-nearest same-affordance node is map-closer than a random one	86%
within-affordance cosine, map-nearest vs farthest 20% of pairs	.29 / .21

Table 8: Superposition statistics for the 4516 diff-means intersection directions at layer 18. The directions occupy a small subspace and overlap far more than chance; the overlap is organized by affordance (the set of legal moves) and then, within an affordance, by map position.

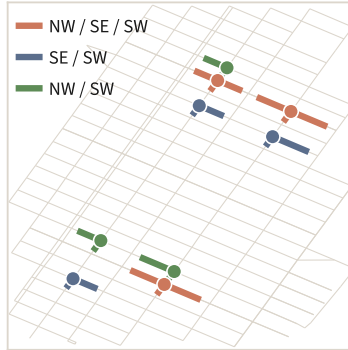


Figure 10: Affordance families on the real map, in one neighbourhood. Each colour is a group the model superposes: an intersection and its two nearest same-affordance direction-neighbours, strongly aligned with it (cosine > 0.75) though at different places on the map, all sharing the same legal-move set (an affordance). Each intersection’s affordance is drawn as its legal outgoing streets (these are one-way, so it is the set of legal exits, not every street that meets the intersection).

F LOCALIZING: STATE-TRACKING VS RE-DERIVATION

We KV-edit to see what the final token’s current-intersection decode (diff-means, layer 18) relies on. At every block we edit each past token’s residual and recompute its key and value, then forward only the final token against this edited past; history’s own forward is untouched, and with no edit the clean decode is reproduced. To remove past *moves* we subtract the diff-means of the eight moves, built the same way from move-token residuals; this drops move decode to chance and leaves the current-node decode intact (Figure 2a, $-$ moves). So the current position is not carried by the past moves.

To remove past *positions* we *ablate the intersection code*: along the intersection-code subspace (the top PCA directions of the node diff-means holding 90% of their variance, ≈ 120 dims per layer) we

set each past token to its *move centroid* – the mean residual of a token making that move, which has the position averaged out – and leave the orthogonal complement clean. This removes the stored position while keeping the move and everything outside the subspace, and it is principled rather than tuned: its ceiling is the full move-centroid replace (every past token set to pure move, which removes *all* position and reaches decode 0.16 overall), and the intersection-code ablation reaches the same 0.16 while touching only the intersection subspace. It is stable in the subspace size (identical from a few hundred dims up) and a matched random subspace of equal dimension leaves the decode at baseline (Figure 2a, random control 0.998).

Ablating the intersection code collapses the current-node decode (Figure 2a, $-\text{pos}$: 0.15 at 27–34 hops against a baseline of 1.00), so the model localizes by *state-tracking* its stored past positions. What survives is a weak re-derivation from the origin and moves: it works on short walks and fades as the walk lengthens (0.69 at 4 hops down to 0.14 at 50; Figure 11). Revealing only the k most-recent past positions and decoding against k shows the model integrates a *look-back window* of several positions, not just the last (Figure 2a, inset).

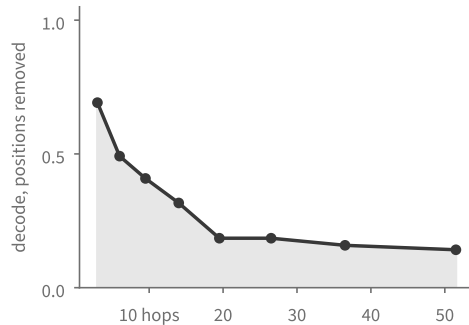


Figure 11: Current-node decode with the intersection code ablated from the past, by walk length: the re-derivation backup from origin and moves is high on short walks and fades as the walk deepens.

G THE GOAL COMPASS

To choose a move the model must know which way its goal lies, and it keeps a direction in the residual stream that says so: at each step, a needle pointing from the current intersection toward the goal. We call it the *goal compass*. We first came across it through a sparse autoencoder on the residual stream, where a handful of features tracked the goal’s bearing; the SAE gave little else usable, but it pointed us here, and the diff-means compass below is the clean version.

Decode. At each mid-walk state the true bearing to the goal is its angle on the map. We decode that angle from a pair of directions, one for its cosine and one for its sine, built by diff-means: split the bearing into 16 slices, average the residual within each slice, and combine the slice means into a cosine axis and a sine axis. Subtracting each intersection’s own mean first leaves the axes carrying the goal direction, not which corner the taxi sits on; states within three hops of the goal are dropped, where the bearing is ill-defined. The axes are fit on 6,000 training walks ($\approx 121\text{k}$ states) and evaluated held-out on 3,000 walks.

Decoding a held-out state recovers the goal’s direction to a median 18° at layer 16, and near that across mid-network (Figure 12a; Table 9). On the two axes the states fan into a ring whose angle matches the goal direction (Figure 2b).

layer	10	12	14	16	18	20	22	24
median error (deg)	25.6	22.6	20.1	18.1	19.0	19.4	20.5	21.3

Table 9: Median error (degrees) between the true goal bearing and the one read off the two axes, held-out, per layer. Best at layer 16.

Causal steering. The compass is causal: plant a direction and the taxi drives that way. At each step we subtract β times the state’s own two-axis reading and add a needle at θ with amplitude $\alpha\|h\|$. The model then navigates greedily (best legal move) for 12 steps, and its *net heading* is the bearing from its start to the intersection reached after those 12 moves, one direction per walk. We score steering by $\cos(\text{net heading} - \theta)$ over walks 6–30 hops long: 1 toward θ , 0 at a right angle, -1 opposite (a cosine, not a percentage); 0.92 is about 23° off. Read as an angle, and pooling every steered walk rather than averaging within a bearing first, a walk’s own heading lands a median 18.4° from the planted bearing, a mean of 24.9° , with quartiles at 8.6° and 33.5° over 1,466 walks: most follow closely and a tail wanders. Some bearings are followed far better than others, best at 68° (median 6.5°). None of this is a limit of the streets: taking every intersection reachable in twelve legal moves, the best of them sits a median 0.9° from any bearing we could ask for, so the scatter is the model’s own.

Steering peaks near $\alpha = 1$ and collapses by $\alpha = 2$, where the edit swamps the residual; erasing the model’s own reading (β) helps only when the injection is weak, and $\beta > 1$ adds nothing (Table 10). We use $\alpha = \beta = 1$ at layer 16: alignment 0.92 on this grid’s sample and 0.85 averaged over the sixteen planted bearings below, the two differing only in which held-out walks they draw. Steering barely varies across mid-network layers. For the north/south illustration in Figure 2b we instead *hard-clamp*: at layer 18 (which tolerates a strong overwrite where layer 16 collapses) we drive the compass to due north or south at $\alpha = 16$ for a stretch of the walk, so the taxi is forced onto that heading and zigzags the diagonal street grid, then recovers its true goal when released.

	$\beta=0$	$\beta=0.5$	$\beta=1$	$\beta=1.5$	$\beta=2$
$\alpha=0.25$	.716	.803	.849	.831	.838
$\alpha=0.5$	.866	.875	.887	.880	.875
$\alpha=1$	.913	.921	.922	.923	.920
$\alpha=1.5$	.889	.892	.880	.872	.858
$\alpha=2$	.538	.554	.541	.512	.489

Table 10: Steering alignment over the strengths (α, β) at layer 16. We set $\alpha = \beta = 1$.

Across 16 planted directions (Figure 12b) the taxi follows best along the street-grid axes (0.93–0.95) and worst on the off-grid diagonals (0.72–0.79). The response is graded rather than a few fixed settings: swept around the circle, the net heading turns with θ one-to-one.

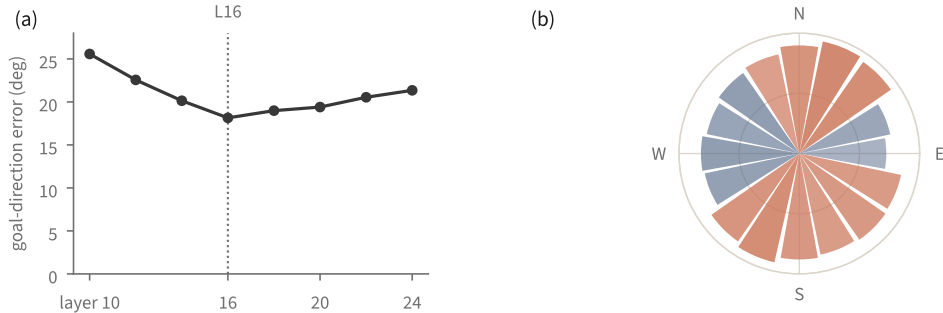


Figure 12: The goal compass. (a) median goal-direction error per layer (held-out), best at layer 16. (b) steering alignment for 16 planted directions (longer bar = closer): best along the street-grid axes, weakest on the diagonals.

H NAVIGATING: TWO MECHANISMS IN THE MOVE LOGITS

Two mechanisms write into the eight move logits (Figure 2b). They are not the whole story (the stop feature of App. I is another, and smaller effects surely remain), but between them they account for the move the taxi takes. The intersection’s own affordance scores its legal moves high and its

illegal moves low, whatever the goal. The goal compass adds a preference for moves that point toward the goal, whatever their legality. The affordance writes an order of magnitude more strongly than the compass (± 12 against ± 1), but the compass still decides which move is taken. Because the affordance is nearly the same for every legal move, the preference among the legal moves comes almost entirely from the compass (95% of it).

Measuring the two writes. The affordance write is a logit-lens: we send the current node’s diff-means direction (the same one that reconstructs the graph, App. C) through the unembedding at layer 32 and read its eight move logits, which score legal moves high ($\approx +12$) and illegal low (≈ -4) whatever the goal. The compass write we read causally: at layer 16 we swap the compass to point at the goal versus away (subtract its current in-plane reading, add a unit compass at strength $\alpha \|h\|$ with $\alpha = 1$) and take the swing in the output move logits ($\approx \pm 1$). Each of the eight moves is labelled by its real-map bearing to the goal, *goalward* within 60° and *awayward* beyond 120° (the middle band excluded); the labelling is geometric, so it applies to the illegal moves too. Averaging each write over the four goalward/awayward $\times$ legal/illegal cells gives the values in Figure 2b.

We test this by ablating the compass, projecting its cosine and sine directions out of layer 18. The model still picks a legal move as often as before, and as often as under a matched random-plane control, but the legal move it picks changes at 33% of states. Over a full walk it keeps moving legally but reaches the goal much less often, and the gap grows with the starting distance, while the random-plane ablation has no effect (Table 11, Figure 13).

start distance (hops)	3–7	7–12	12–18	18–32
<i>reaches goal:</i>				
with compass	1.00	1.00	0.99	0.94
compass ablated	0.68	0.54	0.38	0.23
random-plane ablated	1.00	1.00	0.98	0.94
<i>moves that are legal:</i>				
compass ablated	0.995	0.997	0.996	0.995

Table 11: Self-generated walks (Vafa’s protocol), by start distance. *Top*: the fraction reaching the goal. Ablating the compass collapses this, the more so for far goals; a matched random 2-plane does nothing. *Bottom*: the ablated taxi is free to sample an illegal move and almost never does (legal-move rate ≥ 0.995 in every bin, against 1.000 for the baseline). It keeps moving legally even where it seldom reaches the goal (0.23 at the far bin).



Figure 13: Four compass-ablated walks on the real map (hand-picked real examples). With the compass removed the taxi keeps making legal moves but wanders and never reaches the goal. Origin = circle, goal = star. The baseline reaches every one.

I THE AT-GOAL FEATURE: STOPPING

The model ends a walk by emitting a stop token when it reaches the goal. We isolate the feature that reads “am I at the goal?”. At a reached state (origin, moves, current node C) the position is computed from the origin and the moves and is independent of the goal token, so we swap the goal: goal = C (at the goal) against a node at least 15 hops away (not). The diff-means of the two, per

layer, is the at-goal direction. On held-out states the feature fires precisely when the taxi is at the goal and stays quiet otherwise: its projection tells at-goal from not almost perfectly across the mid-network, best at layer 20 (Figure 14a), matching the model’s own stop probability ($P(\text{end}) = 0.55$ at the goal against 0.00 away from it).

It is causal. Injected at a not-at-goal state, where the model would keep going, the at-goal direction makes it emit the stop token, while a matched random direction does nothing (Figure 14b, Table 12): at a strength of $\alpha = 1$ (α = fraction of $\|h\|$ added at layer 20) the stop rate is already 0.98, reaching 1.00 by $\alpha = 2$. ($P(\text{end})$ is the probability the model puts on the stop token; the stop rate is the fraction of states where the stop token is its argmax, i.e. where it would actually stop.)

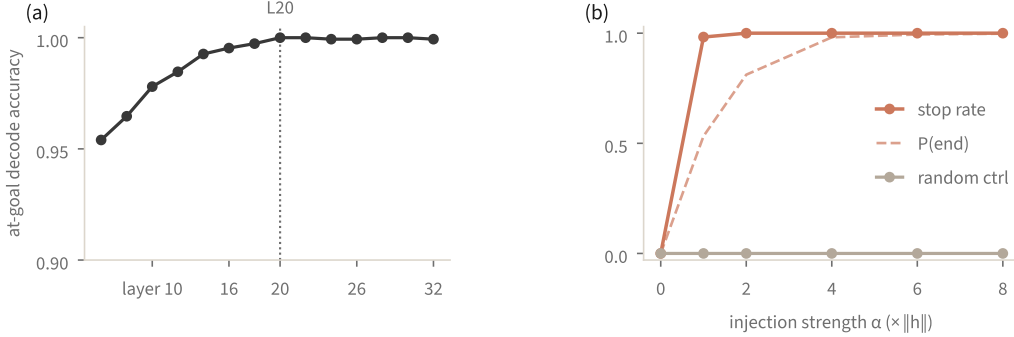


Figure 14: The at-goal feature. (a) held-out accuracy of decoding whether the taxi is at the goal, across layers, best at layer 20. (b) injecting the direction at not-at-goal states drives the stop token (stop rate and $P(\text{end})$ climb with strength), while a matched random direction leaves it at zero.

strength α	0	1	2	4	6	8
at-goal: stop rate	0.00	0.98	1.00	1.00	1.00	1.00
at-goal: $P(\text{end})$	0.00	0.53	0.81	0.98	0.99	1.00
random: stop rate	0.00	0.00	0.00	0.00	0.00	0.00

Table 12: Injecting the at-goal direction at not-at-goal states (baseline stop rate 0): the model emits the stop token, while a matched random direction never does.

A genuine detector, not a stop-writer. The model has a second route to stopping: at its ≈ 99 -move training length it emits the stop token regardless of position (a *forced* end). This separates “am I at the goal?” from “am I about to emit end?”. Over self-generated far walks, the at-goal feature fires at genuine arrivals but stays quiet at the forced ends, even though the model emits the stop token in both (Table 13): it reads arrival, not the stop token. (We count the feature as *firing* at a state when its layer-20 projection exceeds the midpoint threshold that separates at-goal from not in the goal-swap decode above.)

final state	at-goal feature fires	$P(\text{end})$	n
genuine arrival (stops at the goal)	0.93	0.80	2411
forced end (stops at training length, not at goal)	0.13	0.89	383

Table 13: The at-goal feature fires at genuine arrivals but not at forced max-length ends, though the model emits the stop token ($P(\text{end})$ high) in both. So the feature reads arrival, not the stop token.

J THE COMMIT-TO-GOAL FEATURE

The model reproduces two habits of its random-walk training: it wanders rather than going straight, and it steps over the goal and loops before stopping (overshooting 26–44% of its own walks). One direction governs both.

We set out to isolate a *no-stop* feature. At *at-goal* states (current node = goal) the walk either stops or steps over and continues; the diff-means of continue versus stop, over genuine early overshoots, decodes at 0.84 (layer 16), and injecting it where the model would stop makes it refuse ($P(\text{end}) = 0.61 \rightarrow 0$; Figure 15a). But the direction is not only about stopping. Steering it also dials how directly the taxi drives to the goal (Figure 15b): toward the goal it pulls the self-generated path from $\approx 3.5\times$ the shortest-path length down to $\approx 1.3\times$ and stops there; the other way it wanders and loops. It is a *commit-to-goal* feature, dialing the model between a shortest path and a full random walk in both directions, with reach held through the commit side and falling as it is steered to explore; on the map the route grows from a near-direct line to a loop (Figure 16).

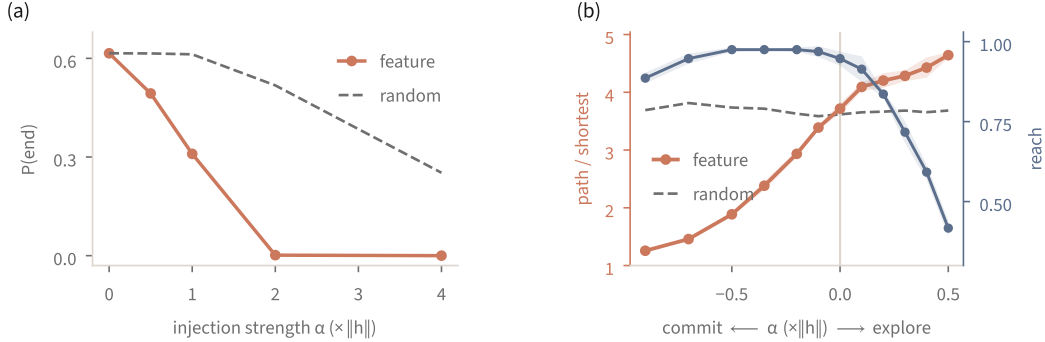


Figure 15: The commit-to-goal feature. **(a)** Injecting it at at-goal states where the model would stop drives $P(\text{end})$ to zero, so the model refuses to stop; a matched random direction does nothing. **(b)** Steering the same direction dials the self-generated route: the path over shortest-path length (clay) runs from near-shortest to a full random walk, while reach (slate) stays high across the commit side and falls as the feature is steered to explore. Steering strength is α in units of $\|h\|$. Bands are the standard deviation over three generation seeds; a norm-matched random direction (dashed) leaves the path length flat, so the dial is specific to this direction.

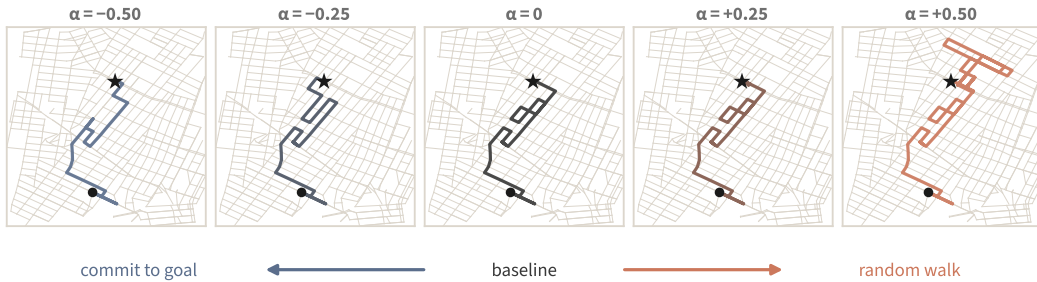


Figure 16: The commit-to-goal feature on the real map, a hand-picked real example: the same origin (circle) and goal (star) at five steering strengths (α above each panel), from strong commit (near-direct, and it stops) through baseline to strong explore. The mild-explore panel ($\alpha=0.25$) looks almost like the baseline, but on a close look it loops once around the destination instead of stopping.

A direction found specifically for the meandering, by diff-means on non-shortest moves or by gradient descent for shortest paths, was weaker and one-sided, so we report only this one.

The model uses this direction during its own generation, and engages it more the deeper into a walk it gets (Figure 17). We read this as a wrap-up heuristic: training walks are bounded and always reach the goal, so the longer a walk has run the more the model should stop wandering and head in.

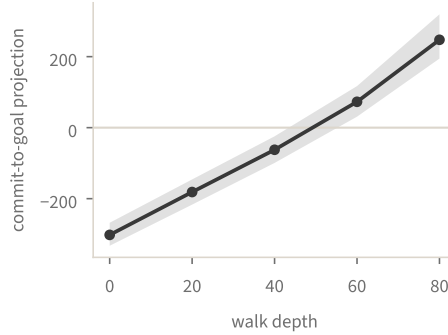


Figure 17: The commit-to-goal feature as the walk runs long (projection signed so up = commit; band is the interquartile range within each depth bin).

K VAFA’S STRESS TEST IS OUT OF DISTRIBUTION

Vafa samples origin–destination pairs uniformly from those seen in training with a legal traversal under the length cap, and has the model self-generate a walk for each at temperature 1. Running that protocol on his exact pairs, our pipeline fails on 19.1% of them, matching his 19.1%. For resolution on the failure modes we generate a larger set of 20,000 walks on distinct pairs, sampled to match the hop-distance distribution of his committed set, which reproduces the same $\approx 19\%$ failure rate.

What takes this out of distribution, on the face of it, is distance. Training pairs sit a median 9 hops apart; the stress pairs sit 32. The failure rate climbs with that distance, from near zero to about 0.9 (Figure 18a).

At those distances the model cannot arrive at all, and that follows from its training rather than from any shortcoming of the model. Training gives it two things that do not fit together here. It gives it a budget of 99 moves, and nothing at all beyond that: positional embeddings past ≈ 101 were never trained and still sit near their initialisation (norm 0.66 against 1.6–2.3 in range), so a longer sequence is not something the model can produce. And it teaches it to meander, taking routes about 4.7 times the shortest path. At that rate 99 moves reach a goal about 21 hops away, while 77% of the stress pairs are farther. The task being asked of the model is not compatible with the policy it was trained to follow. It does already wander less when the goal is far, taking only 2.6 times the shortest path on the far pairs it solves, but not nearly enough to close the gap. That leaves the model somewhere training never puts it: deep into a walk with the goal *still* far away, where the moves it has left can no longer cover the distance that remains. When a training walk has run a long way, it is usually close to its goal with few moves still to make (Figure 18b). That combination is 0.1% of in-distribution steps against 5.4% of stress steps, and it costs more than its parts: depth alone leaves the write at 517 and a far goal alone at 422, but together it is 373, about 100 below what the two predict apart. The failures follow. Binning every stress step by depth and distance to the goal, the off-graph rate per ten thousand steps is 0.5 far from the goal but early, 8.4 deep but near it, and 466 where the walk is both deep and still far, so that corner holds 5.4% of the steps and 57% of the off-graph steps.

One might object that a model sampling at temperature one always leaves a little probability on illegal moves, so a long enough walk will draw one whatever the regime, and the off-graph steps say nothing about being out of distribution. It does not. In distribution the mass on off-graph moves at the true node has median 2×10^{-5} , and the model’s most likely move is off-graph 0.009% of the time, at every depth. At the failing steps under stress that mass is ≈ 0.2 –0.47.

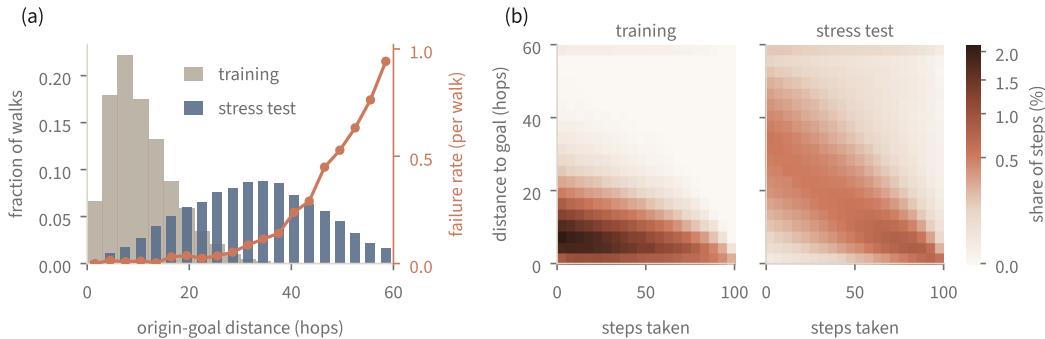


Figure 18: What the stress test changes. **(a)** Origin–goal distance, training (sand) against the stress test (slate): the pairs are far, and the per-bin failure rate (clay, right axis) rises from near zero to about 0.9. **(b)** Where the walks sit, over length and distance to the goal. In training a long walk is almost always a nearly-arrived one; under stress the model is routinely deep into a walk with the goal still far, which barely happens in training.

L WHAT WEAKENS THE WRITE, AND WHAT RAISES THE NOISE

An off-graph step needs a weak write of the true position competing against the broadband noise in the position subspace. We harvest the layer-18 residual at every step of 12,000 stress walks, take the write as its component on the true node’s diff-means direction and the noise as the root-mean-square component on 96 fixed other node directions, and ask what each varies with, holding the others fixed in each case. The two come apart: the noise tracks one thing, and the write tracks several. Except where we say otherwise these are associations, and the factors are related to one another, so we report what each varies with rather than ranking them or claiming that one causes another. Taken together they read less like a store that has been damaged than like a confidence that has been eroded, a reading we offer rather than test.

The noise is mostly noise. Once the correct direction is removed, what is left of the position code sits at every angle to it in roughly equal amount: across angle bins the strongest reads 49 and the weakest 44 at failing steps, and 28 against 23 on clean ones. Nor does it pile up around the taxi. The intersections within three streets of it read 15% above the rest of the map at failing steps and 25% above on clean ones. That lean is weaker at failures than on clean steps, which is the wrong way round for a position code spreading outwards.

The noise grows as the walk runs long. Over a walk the per-direction noise rises from ≈ 49 to ≈ 71 , and nothing else we measured moves it. It does the same in ordinary navigation, rising from ≈ 50 to ≈ 66 on held-out walks, so this is not something the stress test induces. At a failing onset the sudden change in the residual is close to isotropic, its top principal component accounting for about 5% of it, so the noise is a standing backdrop rather than a single direction that fires. Depth does not weaken the write: at a fixed distance to the goal a walk that has run longer writes its position slightly *more* strongly, and length costs the write only in combination with a distant goal (App. K).

The write is weakest where the map is most superposed. Nodes differ in how many others sit within a small angle of them in the position code ($\cos > 0.5$). The write falls steadily as that number grows, from a median of 531 at nodes with a single close neighbour to 398 at nodes with around twenty (Figure 19). The position is hardest to write exactly where it is least separable from its neighbours. This is a property of the node rather than of the route taken to reach it, and it is not simply the number of streets at the intersection, which does not carry the effect.

The write is weaker the further the goal still is. The write falls with distance to the goal, from a median of about 510 one hop away to 433 far away. This is not an artifact of when in a walk those states occur: states far from the goal are mostly early states and the write is genuinely lower early, but binned within each band of walk length the fall survives in every band, and on held-out walks

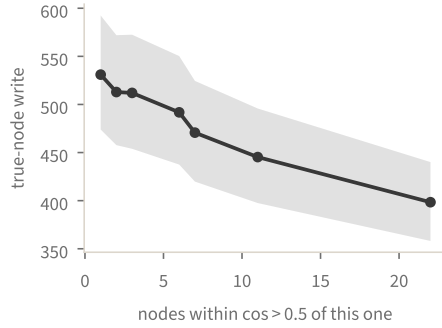


Figure 19: The write against how superposed the current node is: the number of other nodes lying within a small angle of it in the position code. Median with the interquartile range over states in each bin. The position is hardest to write where it is least separable from its neighbours.

as well as under stress (Figure 20). It is also causal, which we can test because the goal is only a token in the prompt. Replaying 800 stress walks from the same origin with the same moves, and changing only that token, a goal about four hops from where the taxi now stands gives a write of 496 and one about forty-five hops away gives 434, against 468 for the original goal; the nearer goal is the stronger on 86% of walks. Some of that gap is indirect, since under a distant goal the moves actually taken are ones the model rates less likely (-0.79 against -0.68 in mean log-probability) and an unexpected route weakens the write by itself, which accounts for about 11 of the 62 points. The rest is the distance. The model writes its position less strongly while the goal is far and more strongly as it closes on it, in ordinary navigation as much as under stress. We hypothesise that this is a strategy of the model’s own rather than a limitation it suffers, but we were not able to uncover what it is for.

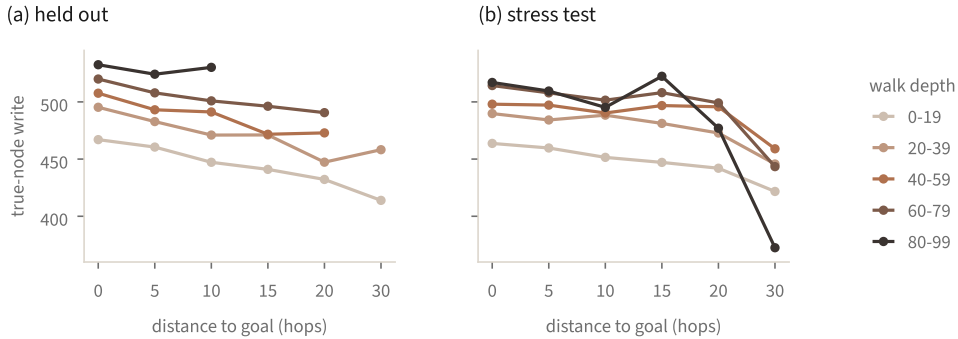


Figure 20: The write against distance to the goal, within bands of walk length, on (a) held-out walks and (b) the stress walks. It falls with distance in every band and in both, so it is goal-conditioning rather than an artifact of when in a walk those states occur. Deeper bands sit higher: at a fixed distance to the goal, depth *strengthens* the write. The held-out curves stop where the combination stops occurring, a training walk never being both deep and far from its goal; under stress it does occur, and the write there falls to 372, well below what the two trends predict on their own.

The write is weaker the less likely the model finds its own route. A route it rates unlikely is one that corresponds less well to the distribution it was trained on, and the write is weaker along it. Over a whole route, the write falls as the mean probability the model gave the moves it has taken falls (Figure 21a). The same effect appears one step at a time: under temperature-one sampling the model sometimes takes a move it rated unlikely, and the write at the node that move leads to drops in proportion to how unlikely it was, from -50 after the least likely moves to $+15$ after the most confident (Figure 21b), recovering over the next two or three steps. Aligned to the event, the write is

520 at the departure node and 476 at the node the surprising move leads to, against a length-matched confident control that does not dip ($503 \rightarrow 510$). Both measures are specific to the write, the same measures against the noise being flat, and both select on the state as well as the sample: the chance of drawing an unlikely move is itself higher where the model’s move distribution is flat.

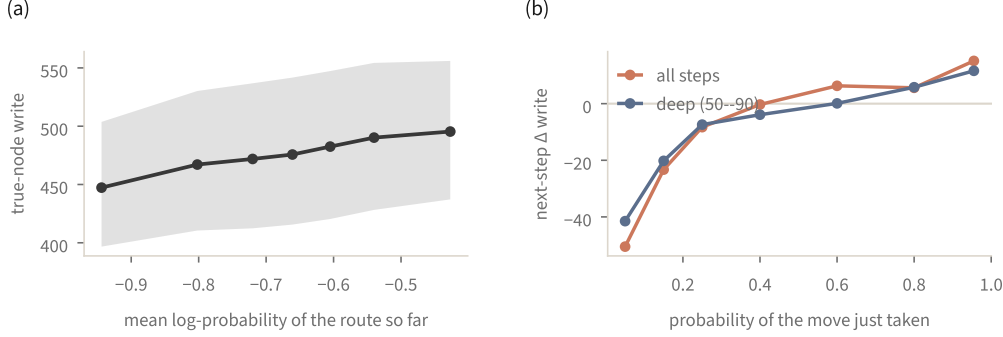


Figure 21: The write is weaker the less likely the model finds its own route. **(a)** The write against how likely the model found the moves it has taken so far, with the interquartile range over states in each bin. **(b)** The same effect one step at a time: the change in the write at the next step, against the probability the model gave the move it just took, over all steps (clay) and within a deep band (slate). The matching curves on the broadband noise are flat and are not plotted.

What does not seem to weaken the write much. We fit a standardized coefficient for several other things one might expect to matter, each with the rest held fixed. None of them registers: how often the route changes direction, how directly it covers ground, how often it returns to a node it has already visited, whether it has been heading away from the goal, where the taxi sits on the map, and path aliasing, distinct routes whose last few moves are identical, all come out at $|\beta| \leq 0.03$, against $+0.55$ for the write at the previous step and -0.20 for superposition on the same fit. We take that as a sign they contribute little here.

M THE FOUR KINDS OF ILLEGAL MOVE

Section 3.1 names four kinds of illegal move and gives their shares, and Figure 4 shows how much each edit heals. Neither says how a given case is assigned to a kind, or what the healing bars actually measure. Both are here. Throughout, the model’s position means the node whose diff-means direction the layer-18 residual lies closest to.

How we sort them. Each illegal move falls into one of four kinds by the rule in Table 14. It uses three quantities: how far the residual lies along the give-up direction u_{stop} , which is the diff-means of wrong-end against deep successful steps with the position subspace projected out and whose threshold is a twentieth of the level it reaches at a genuine stop (App. N); which node the position reads as; and how close that node is to the true one. Two quantities that are *not* used move with it anyway, which is what makes this more than a labelling. The write falls from 504 on clean deep steps through 340 on silent slips to 236 on fatal ones and 171 on full corruption, with non-overlapping bootstrap intervals between the slip kinds. The shared affordance falls the same way, from 1.0 on recovering slips to 0.25–0.33 on silent and fatal and 0 on full corruption. Fitting the give-up direction and its threshold on one half of the walks and sorting with them moves the slip shares by a point or two and the give-up share by four, so the rule is not leaking through its own fit. The boundary between fatal and full corruption is soft, and the give-up share is the one that turns on where its threshold sits: doubling the cut roughly halves it, and at a quarter of the genuine-stop level the category empties, about half of its cases reappearing as full corruption and the rest split between the slip kinds. The threshold itself is set once, on the stress test, and used unchanged for the detour walks; where it is put and why is in App. N.

Table 14: How each illegal move is sorted. The conditions are taken in order, on the layer-18 residual v with the global mean removed, where u_n is node n ’s diff-means direction and the model’s position is $\arg \max_n v \cdot u_n$.

kind	condition	share
give-up	$v \cdot u_{\text{stop}} \geq \tau$	16%
silent slip	else the position is the true node T	25%
fatal slip	else it is a node $L \neq T$ with $u_L \cdot u_T > 0.2$	28%
full corruption	else it is a node $L \neq T$ with $u_L \cdot u_T \leq 0.2$	24%

What the healing bars measure. The failures are sampled at temperature one, so whether the most likely move is legal is a brittle score: a case can flip on a hair, and a large drop in the probability of the illegal move counts for nothing. We measure instead the probability mass the model puts on off-graph moves at the true node, and report the pooled heal $= 1 - \sum P_{\text{illegal}}^{\text{after}} / \sum P_{\text{illegal}}^{\text{before}}$ over 300 cases per kind. The three bars in Figure 4, row 3, are three edits at layer 18: remove the single most active co-active node, remove all position content other than the true node’s, and restore the write to the level of a clean step.

A control for those bars. To check that a heal comes from the edit itself rather than from disturbing the residual at all, we repeat each edit in a matched but meaningless form. Removing a random co-active node, of the same norm as the most active one, heals nothing (-0.03 to 0.01); adding the same amount of norm along a random direction rather than the true node’s heals 0.00 . So both edits act through the particular node and the true-position direction.

The position the model reads is what chooses the move. A weak write and noise put the model on a wrong node; this asks what follows from being there. Read from the residual, the emitted move belongs to the node the model landed on, but that could be a coincidence of measurement. So we put the position where we want it. On clean deep states that read the true node T and emit a legal move, steering the residual toward a co-active wrong node T' makes the model emit T' ’s own move, off-graph at T . The rate rises with the strength of the steer, from 0 to 0.18, 0.42 and 0.48, and at every strength the illegal-move rate equals the rate at which the emitted move is exactly T' ’s, not merely some off-graph move. Steering toward the true node induces nothing at any strength, and a random direction of the same norm does nothing until pushed to twice T' ’s scale, where it reaches 0.12. The node the model takes itself to be at selects the move that comes out.

The illegal move need not come from the most active node. For silent slips the emitted move is legal at the single most active non-true node 54% of the time, against 32% for a node drawn at random, and the gap closes as more nodes are admitted until twelve co-active nodes and twelve random ones are almost equally likely to contain it (Figure 22). The leak comes from the nodes the model is actually activating, but several of them contribute and a rule that scanned deep into the bag would be counting coincidences.

The look-back window is part of why slips correct themselves. The model re-derives its current node each step from a window of recent positions, so we can ask how much of the recovery that window accounts for by taking it away. Revealing only the K most recent positions and averaging away the older ones, a slip returns to the true node 46% of the time on a three-position window, against 79% for a matched clean step, and only the full window brings both to 100% (Figure 23). A slip needs more of the window than a clean step does, so the look-back is doing part of the work of pulling it back.

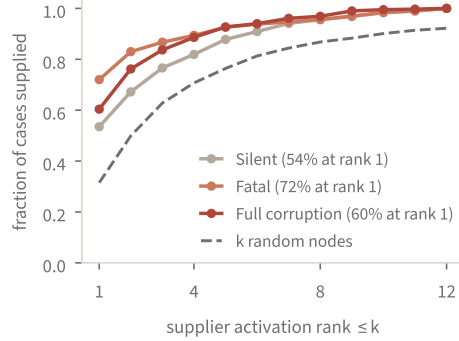


Figure 22: The emitted illegal move is carried by the co-active nodes the model activates most, far more often than by nodes picked at random. Cumulative fraction of illegal cases whose shallowest affordance supplier sits at activation rank $\leq k$, against k nodes drawn at random (dashed). At rank 1 the gap is large (54% for silent slips and 72% for fatal ones, against 32% by chance) and it closes as k grows, so it is the most active nodes that carry the move and the deeper bag adds little beyond chance.

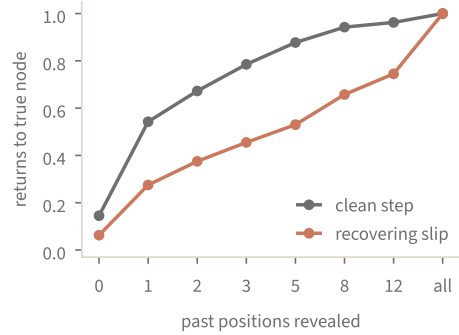


Figure 23: Recovery leans on the look-back window. Fraction that returns to the true node as we reveal only the K most-recent past positions and ablate the older ones. A recovering slip needs more of the window than a matched clean step, and only the full window brings both back.

N THE GIVE-UP FEATURE INTERFERENCE

The feature. One of the four kinds is not a superposition slip. In the give-up category, 16% of illegal moves, a separate feature fires hard and the illegal move falls out of a flooded residual rather than being chosen. We find the feature by taking the mean layer-18 residual at the final step of walks that stop away from their goal, subtracting the mean at deep steps of walks that arrive, and projecting out the position subspace; the same vector is used everywhere it appears in this paper. It is not an artefact of that particular contrast: extracting it instead from the onset of belief divergence recovers it at a cosine of 0.96, and contrasting stopped walks against *arrived* walks at their final step, so that only the outcome differs, at 0.94. Nor is it the at-goal feature of App. I, with which it shares a cosine of 0.21.

Steering it makes the model stop. The direction aligns with the *end* token more closely than with any other direction in the vocabulary, and adding it to a clean deep state flips the very next token to *end*, dose-dependently and from any point in a walk, while a random direction of the same size never does (Figure 24a).

It fires on giving up, not on stopping. Through a walk the feature is near-silent — its median projection mid-walk is -1.8 — and it barely tracks how far the walk has run (correlation 0.17) or

how many moves are left. It is not positional either: a fit to map coordinates accounts for under 1% of it. What it responds to is how the walk ends. At the last step of a walk that stops away from its goal it reaches 773; at the last step of one that arrives, 53. It is a trigger rather than a timer.

Where the cut sits. Over ordinary navigation the projection averages 2.0, and at a genuine stop it reaches a median of 775. A failing step never approaches that level, so the threshold has to be a small fraction of it rather than a large one: we take a twentieth, $\tau = 38.8$. Half of the raw difference lies inside the position subspace and does no work — steering it raises the position noise by exactly what the added vector accounts for arithmetically, with nothing computed on top — so projecting it out costs no signal (35 healthy standard deviations of separation against 20) and keeps the cut independent of the write and the noise, which App. M holds out as evidence rather than using to define the categories.

The flood arrives with the feature. At a failing step the part of the residual orthogonal to the position subspace nearly doubles while the position part grows much less (Figure 24b), which through the layer norm leaves the position code a smaller share of what is there and gives the give-up column the highest position noise in Figure 4. Driving this direction alone does not reproduce that: added to clean deep states at strengths up to the level of a genuine stop, it leaves both the write and the broadband noise where they were, so the flood is not the work of this one vector. We read the two together as one give-up mechanism, which both activates the feature and floods the stream; the illegal move is then a temperature-one draw from the blurred move distribution that results.

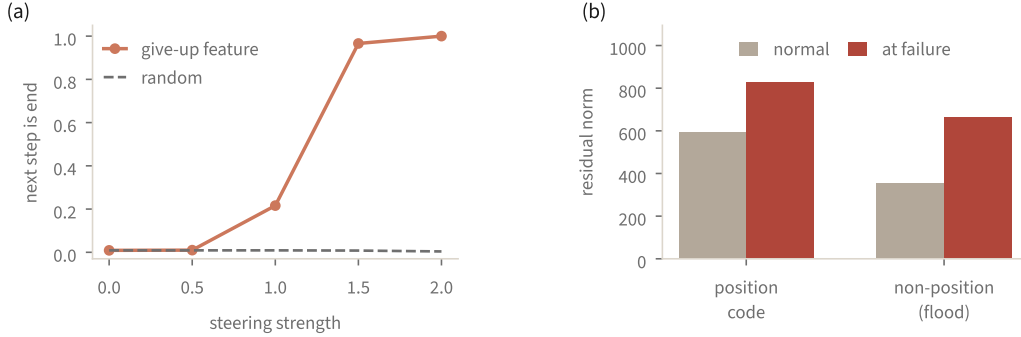


Figure 24: The give-up feature. (a) steering the residual at states sampled across depth and reading the next token: the give-up feature flips it to *end* dose-dependently, saturating at 100% in a single step by $\alpha = 2$, while a random direction of equal norm never does. (b) at a failing step the residual floods: the component orthogonal to the position subspace (the non-position flood) nearly doubles while the position component grows far less, leaving the position code a smaller share of what is there.

O DRIVING THE FAILURES LOWER

Handing the model its position keeps it on the graph. We regenerate the whole stress test with help applied at every step of generation and count the walks that still contain an off-graph step. The control is the same 1,500 origin-goal pairs generated in the same harness with no help, since the help perturbs generation and a comparison against previously recorded walks would confound the two. Adding the true node’s direction to the residual at layer 18, at its own magnitude, takes walks with an off-graph step from 8.9% to 1.2%, and the rate per step from 0.00115 to 0.00014. Adding the same amount of norm in a random direction does nothing (9.1%, 0.00117 per step), so what matters is where the vector points rather than what it adds.

It does not make the model a better navigator. Success falls from 82.0% to 67.1%, all of it into stopping at the wrong intersection. Knowing where you are is not knowing when to stop.

Steering the model to commit trades meandering for directness. The commit-to-goal feature (App. J) At half the direction’s own scale the walks shorten from 78 steps to 47 and off-graph walks fall to 5.7%, but the rate per step is unchanged (0.00120 against 0.00115): the walk is safer only because there is less of it. Stronger steering is destructive, 17.1% at full scale and 71.1% at twice it, where the taxi leaves the graph immediately.

Together they bring both failures down. At full position and half commit, off-graph walks fall to 1.3% while success rises to 89.8% and wrong-end stops return to where they started. The steer supplies what the injection costs. Both failures come down at once, but only by acting on two mechanisms rather than one.

P THE DETOUR TEST

The protocol is theirs. The model drives greedily, and at each step, with probability 0.75, it is overruled onto the legal move it rates least likely — restricted, as they restrict it, to moves that still leave the destination reachable inside the remaining budget. They also require the model to finish the job itself: a walk counts as a success only when the model emits its stop token standing on the destination. We run 10,000 walks from held-out origin–destination pairs. The taxi succeeds on 63.6% of them, takes an off-graph move on 25.6%, and stops at the wrong intersection on 10.7%. Running their own code unmodified on the same checkpoint gives 0.636 (0.022) over 500 trials, so this is their procedure and not a variant of it; both differ from the 0.51 they report for this setting.

The same four kinds of failure, in different proportions. Running the classifier and the hero-figure pipeline on these walks unchanged gives the four kinds with the same position signatures (Figure 25): full corruption 45.7%, fatal slips 35.8%, silent slips 9.2%, give-up interference 9.2%, with nothing left unexplained. Against the stress test’s 24.3, 27.7, 25.2 and 16.2, the mix moves toward full corruption and away from give-up interference.

A forced move does no damage where it happens. One would expect each imposed move to corrupt the position a little, and the damage to add up. It does not. Matched on how far the walk has run, the write at a forced step is the write at a step the model chose itself (Figure 26a). Forcing harder makes the model fail less, not more: at $p=1$ almost every move is imposed, and imposed moves are legal by construction, so off-graph moves fall to 5.4% of walks. The failures are the model’s own moves, in the states the forcing puts it in.

What the forcing changes is where the walk spends its time. The least likely move is seldom a move toward the goal, so the taxi is pushed away step after step, out to a median of thirty hops by step sixty, until the reachability rule leaves it only moves that close the distance and it runs home (Figure 27). It does run home: the taxi still arrives on nearly two thirds of the walks, from thirty hops out and with most of its budget already spent, which is generalization of a fairly demanding kind. But the whole walk, the way back included, sits in the regime the stress test only reaches at its worst. Steps that are at least forty moves in and at least twenty hops from the goal are 32.3% of detour steps, against 15.0% of stress-test steps and 0.4% of training steps.

That regime is where the write is weakest, for two separate reasons. The write falls with distance to the goal, and the goal token is what does it: swap the destination and the write moves with it (App. L), so this looks like part of how the model represents position rather than damage. The write also falls with depth, and that one looks like wear rather than design. Detours put the taxi in both at once. On top of it, the route itself is one the model rates as very unlikely — 22.8% of detour steps fall below a mean move log-probability of -1.0 against 4.2% of stress steps, and the worst detour routes reach -11 where the stress test’s floor is -2.5 (Figure 26c) — because a route that keeps taking the least likely legal move is nothing training contained. Over all steps the detour write runs 36 below the stress write (439 against 475), it falls furthest in full corruption (120 against 171), and that kind takes nearly half the failures. Give-up interference goes the other way, falling to 9.2% from 16.2%, because their reachability rule never leaves the taxi out of budget with the goal still far.

The interventions still work, a little less well. Clearing the position content competing with the true node removes 0.86, 0.72, 0.47 and 0.59 of the off-graph probability across silent slips, fatal

slips, full corruption and give-up interference, against 0.93, 0.79, 0.63 and 0.73 on the stress test; writing the true position back removes 0.63, 0.49, 0.65 and 0.45. The same edits made along a direction that names no node remove between 0.005 and 0.09 (the grey rules in Figure 25).

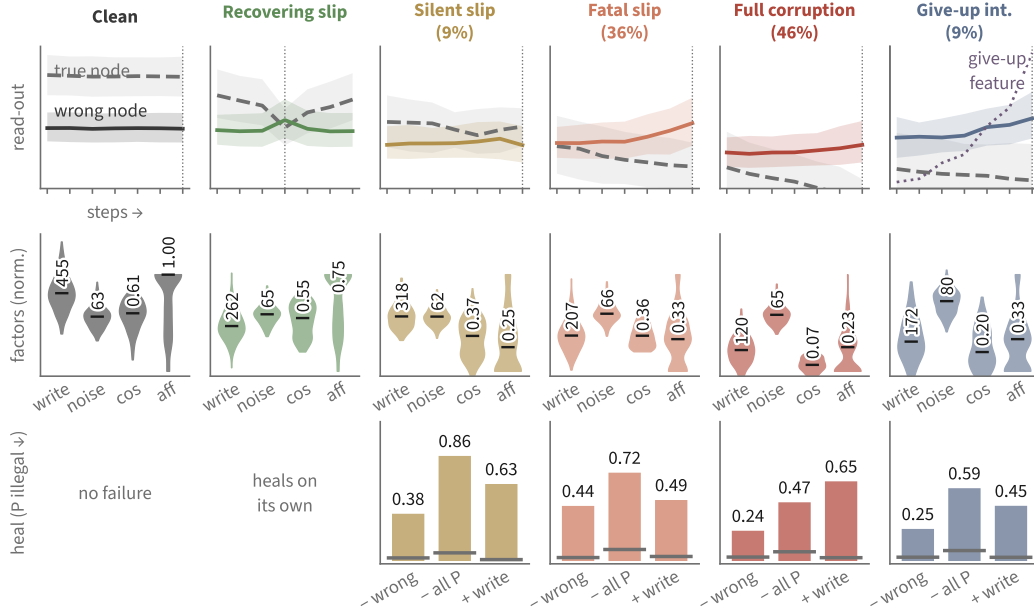


Figure 25: The failure spectrum reproduced under forced detours ($p=0.75$, adversarial), built by the same pipeline as the stress test's. The four kinds show the same position trajectories, factors and healings; the mix moves toward full corruption, the kind the weakest writes produce. The grey rule on each bar is that edit's control.

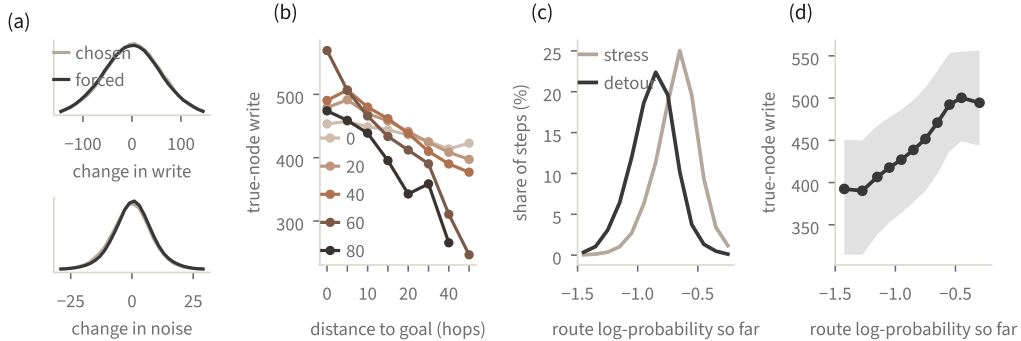


Figure 26: Why the write is weaker under detours, with the give-up failures set aside. (a) it is not the forced move itself: the write (top) and the noise (bottom) after a move are the same whether the model chose it or had it imposed, so nothing is lost at the step where the forcing happens. The damage is in where the walk ends up, and two things about that matter. (b) the taxi is far from its goal and deep into the walk, and the write falls with distance to the goal, steeply so once the walk has run long (bands of walk length, darker with depth). (c) its routes are also ones the model rates unlikely far more often than the stress test's are. (d) and the write falls as they do, at the same rate in both sets, so what changes is how much of the walk is spent there rather than what it costs.

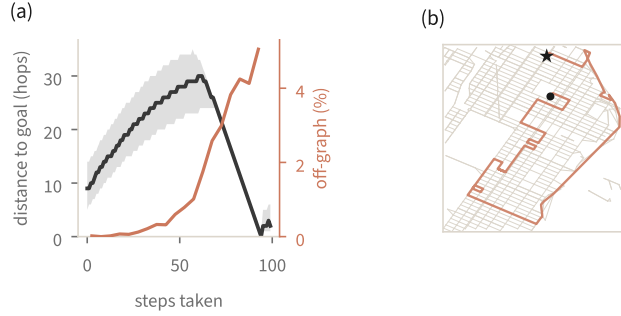


Figure 27: Forcing detours drives the taxi away from its goal, until the rule that the goal must stay reachable brings it home. (a) distance to the goal over a walk, median with the interquartile band: it starts around nine hops, climbs to about thirty by step sixty, then falls as the reachability rule leaves only moves that close the distance. (b) two individual walks, origin a dot and goal a star, chosen only so the route fits the panel.

Q THE COMPRESSION TEST

We replicate the compression test of Vafa et al. at the threshold they use, $\epsilon = 0.01$. It takes one intersection and two routes of the same length that both end there, bound for the same destination; samples thirty continuations from the model after the first route; and asks whether the second route gives every move of every continuation a probability above ϵ (Figure 28). A pair passes only if all thirty agree. Running their code on our checkpoint reproduces their number: 0.479 (0.010) over 2,334 trials, against the 0.50 they report for this model.



Figure 28: One trial. Prefix A and prefix B are two different routes of the same length ending at the same intersection. Thirty continuations are generated after prefix A (four shown), and the test asks whether every move of each still has a probability above ϵ under prefix B. The pair passes only if all thirty do.

Thirty continuations is a demanding test. A trial passes only if all thirty agree, so the score is a per-continuation rate raised to the thirtieth power. With one continuation the model scores 0.95, with five 0.79, with thirty 0.48 (Figure 29); truncating the continuations to five moves gives 0.99 and to twenty, 0.89. So 0.479 is the score of a model that agrees on 97.6% of continuations: it can be right about almost every one and still be reported as compressing half the time. The measured score is better than independence would give — thirty continuations at the one-continuation rate would score 0.24 — because failures cluster on the same pairs rather than falling independently.

The pairs that fail are a little harder, and one of their two routes a little weaker. Their goal is further away, 53 hops against 33, and both routes write the position slightly less strongly than the routes of a pair that passes (Table 15). The agreement the metric asks about is slightly worse too: the two next-token distributions at the shared intersection differ by a total-variation distance of 0.020 where a passing pair gives 0.013. Sorting each pair into its weaker and its stronger member by the write, the two differ by 27 in a passing pair and 31 in a failing one. These differences are all small. Neither route arrives corrupted, and the weaker one does not get its continuations wrong

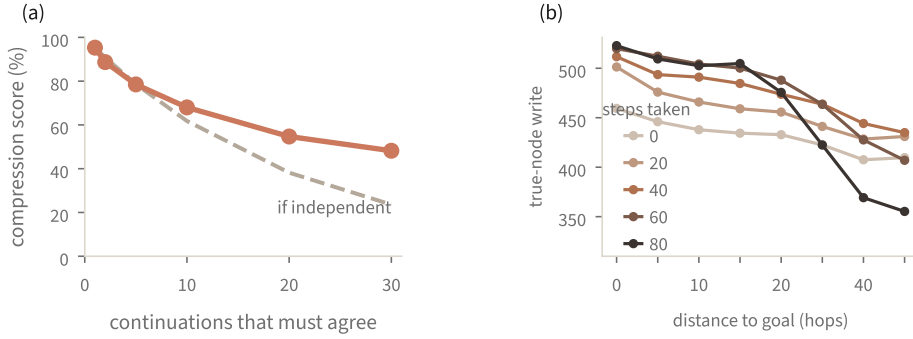


Figure 29: (a) The more continuations a trial has to agree on, the lower the score: one gives 0.95, thirty give 0.48. The dashed line is the score continuations failing independently of one another would give; from ten continuations on, the measured curve sits above it, so failures fall on the same pairs rather than being spread across them. (b) the write on the true intersection over the walks this test generates, against distance to the goal within bands of walk length (darker is deeper into the walk): far from the goal and deep into a walk, the write is weak.

wholesale. Both routes have the same length and the same destination by construction, so neither depth nor distance to the goal separates them, and the route the model rates less likely is not reliably the weaker one.

Table 15: The two routes of a pair, split into the weaker and the stronger by the write. Medians at the end of the route, with quartiles in brackets where a row is a property of the pair rather than of either route — the two members share a length and a destination by construction, so the first two rows span both columns.

at the end of a route	pair passes		pair fails	
	weaker	stronger	weaker	stronger
distance to the goal (hops)	33 (23–45)		53 (40–64)	
length of the route (moves)	21 (9–37)		20 (11–34)	
write on the true intersection	420	454	400	438
noise in the position code	49.7	52.0	49.5	50.8
log-probability of the route	−0.838	−0.835	−0.850	−0.841
a wrong intersection is most active	1.0%	0.3%	1.6%	0.4%

It is the continuation that takes them somewhere hard. The first move to fall below the threshold sits 71% of the way through a continuation of 71 tokens, 19 hops from the shared intersection, at a total depth of 75 moves counting the route that led there. Failure tracks how far the goal is, from 9% of trials at goals within twenty hops to 82% at fifty to seventy, and not the length of the routes, which runs flat at 42 to 56% across every bin. A far goal forces a long continuation, and a long continuation walks the model deep into a walk with the goal still far, where the write is weakest (Figure 29b), and out of the region training ever covered (Figure 30). The routes cannot go there — the test draws them short enough to leave the goal reachable — but the continuations it samples do.

And it rests on the same failures. 28% of the sampled continuations leave the graph. Running the same classifier and hero-figure pipeline over them unchanged gives the same four kinds with the same position trajectories and the same factors (Figure 31), in shares of 22.7, 28.5, 34.6 and 13.7% against the stress test’s 25.2, 27.7, 24.3 and 16.2. Full corruption takes the larger share, as it does wherever the states run deepest.

The map is still there. The same interventions undo these failures at the same rates as on the stress test. Clearing the position content competing with the true node removes 0.92, 0.82, 0.68 and 0.68 of the illegal-move probability across the four kinds, against 0.93, 0.79, 0.63 and 0.73 under

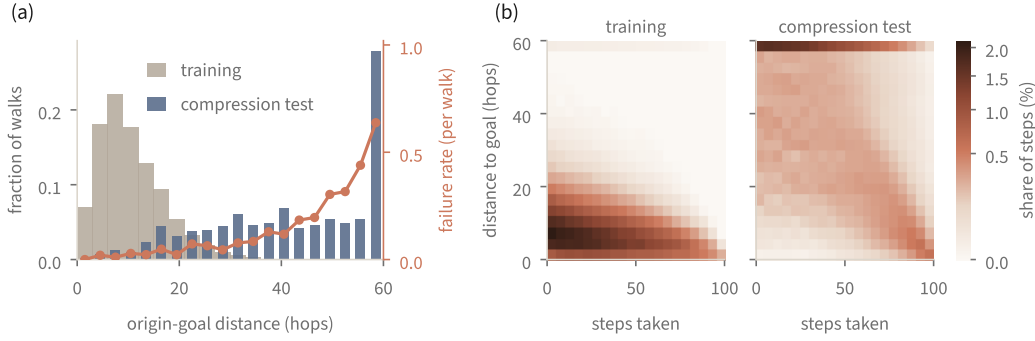


Figure 30: The regime the compression test creates. (a) how far the origin is from the goal: training clusters within about fifteen hops, the test spreads to sixty, and the share of walks that leave the graph climbs with it. (b) where the walks sit as they run. Training stays near its goal, a median of 7 hops over all its steps; the test’s walks sit at a median of 34, with more than half their steps beyond 30.

the stress test; restoring the write removes 0.67, 0.62, 0.73 and 0.56 against 0.61, 0.59, 0.75 and 0.54. So the map is intact and the model can still reach it. Where this test fails, it has caught a model that is not using a map it still has.

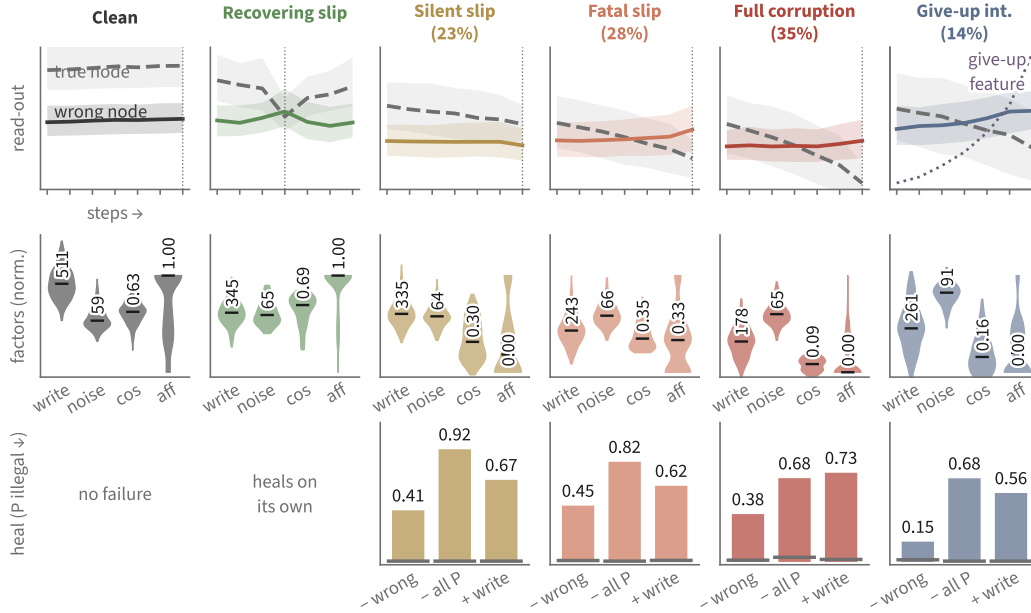


Figure 31: The failure spectrum under the compression test, built over the walks the test itself generates by the same pipeline as the stress test’s. The four kinds carry the same position trajectories, the same factors and the same healings; only the mix moves. The grey rule on each bar is that edit’s control.